Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Advanced Digital Systems Design

ENCS3310

Design and Verification of a Simplified Memory Controller

Prepared by:

Name: Batol Abu Samhadana

ID: 1230738

Section: 1

Instructor: Dr. Ayman Hroub

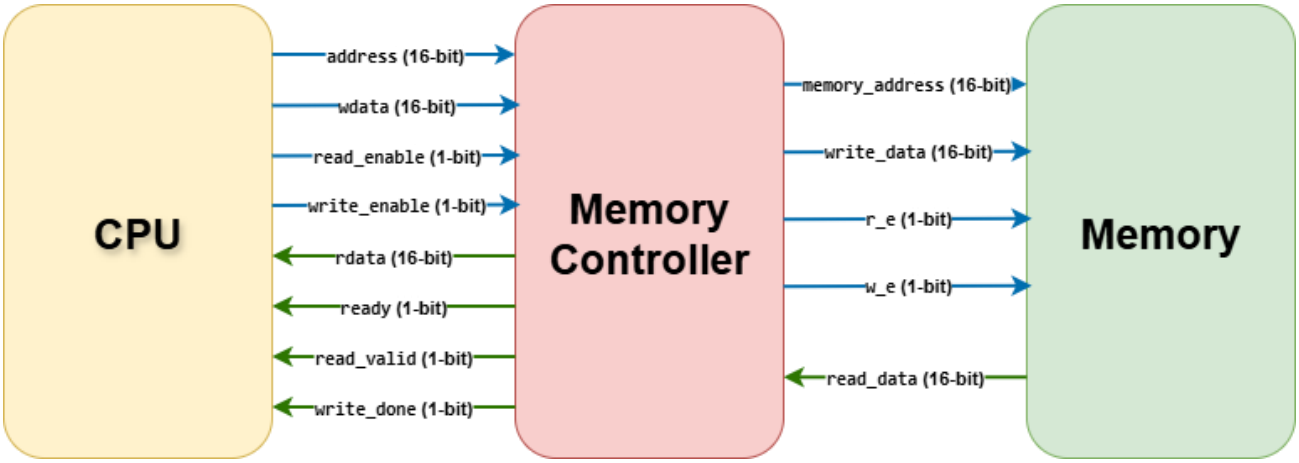
Date: 18/9/2025

Introduction

This project presents the design and verification of a simplified synchronous memory controller that manages data transfer between a CPU model and a memory module. The controller supports read and write operations with write requests having higher priority. It uses a simple FSM with three states—idle, read, and write—to control the flow of operations. A basic CPU model is implemented to generate requests and receive responses, while a memory model stores the data. Handshake signals (ready, read_valid, and write_done) ensure proper synchronization between the CPU and controller. The design is verified using comprehensive Verilog testbenches for both the memory controller alone and the complete system. Simulation results confirm correct functionality across different scenarios.

Table of Contents

System Block Diagram



Memory Controller RTL Code

```
94  module memory_controller (
95      clk,
96      rst_n,
97      address,
98      wdata,
99      rdata,
100     read_enable,
101     write_enable,
102     ready,
103     read_valid,
104     write_done,
105     memory_address,
106     write_data,
107     read_data,
108     r_e,
109     w_e
110 );
111   input clk, rst_n;
112 
113   // CPU interface
114   input [15:0] address, wdata;
115   input read_enable, write_enable;
116   output reg [15:0] rdata;
117   output reg ready, read_valid, write_done;
118 
119   // Memory interface
120   input [15:0] read_data;
121   output reg [15:0] memory_address, write_data;
122   output reg r_e, w_e;
123 
124   // state machine encoding
125   reg [1:0] state, next_state;
126   parameter S_IDLE = 2'b00, S_READ = 2'b01, S_WRITE = 2'b10;
127 
128   reg pending_read;  // flag in case user did read and write together
129   reg [15:0] pending_read_address;
130   reg [1:0] read_cycle_cnt;
131 
132   always @(posedge clk or negedge rst_n) begin
133     if (!rst_n) begin  // reset zero everything
134       state <= S_IDLE;
135       ready <= 1'b1;
136       read_valid <= 1'b0;
```

```
      write_done <= 1'b0;
      r_e <= 1'b0;
      w_e <= 1'b0;
      pending_read <= 1'b0;
      read_cycle_cnt <= 2'b00;
      memory_address <= 16'b0;
      write_data <= 16'b0;
      rdata <= 16'b0;
    end else begin
      state <= next_state;
      read_valid <= 1'b0;
      write_done <= 1'b0;

      case (state)
        S_IDLE: begin
          r_e <= 1'b0;
          w_e <= 1'b0;
          ready <= 1'b1;
          read_cycle_cnt <= 2'b00;
          if (write_enable) begin
            memory_address <= address;
            write_data <= wdata;
          end else if (read_enable) begin
            memory_address <= address;
          end
        end
        S_WRITE: begin
          w_e <= 1'b1;
          r_e <= 1'b0;
          ready <= 1'b0;
          write_done <= 1'b1;
        end
        S_READ: begin
          r_e <= 1'b1;
          w_e <= 1'b0;
          ready <= 1'b0;
          read_cycle_cnt <= read_cycle_cnt + 1'b1;
          if (read_cycle_cnt == 2'b10) begin
            rdata <= read_data;
            read_valid <= 1'b1;
          end
        end
      endcase
```

```
    end
  end

  always @(*) begin
    next_state = state;
    case (state)
      S_IDLE: begin
        if (write_enable) next_state = S_WRITE;
        else if (read_enable) next_state = S_READ;
      end
      S_WRITE: begin
        if (pending_read) next_state = S_READ;
        else next_state = S_IDLE;
      end
      S_READ: begin
        if (read_cycle_cnt == 2'b10) next_state = S_IDLE;
        else next_state = S_READ;
      end
    endcase
  end

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      pending_read <= 1'b0;
      pending_read_address <= 16'b0;
    end else begin
      if (state == S_IDLE) begin
        if (write_enable && read_enable) begin
          pending_read <= 1'b1;
          pending_read_address <= address;
        end
      end else if (state == S_WRITE) begin
        if (pending_read && (next_state == S_READ)) begin
          pending_read   <= 1'b0;
          memory_address <= pending_read_address;
        end
      end
    end
  end
endmodule
```

Memory Controller TestBench Code

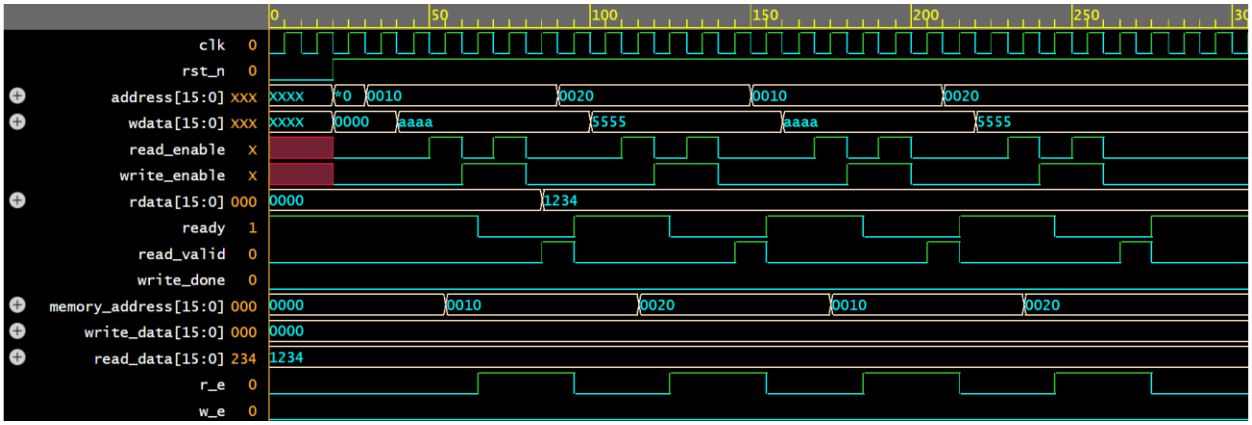
```
`timescale 1ns / 1ns
module memory_controller_tb;

  reg clk, rst_n;
  reg [15:0] address, wdata;
  reg read_enable, write_enable;
  wire [15:0] rdata, memory_address, write_data;
  wire ready, read_valid, write_done;
  wire r_e, w_e;

  initial begin
    $dumpfile("memory_controller_tb.vcd");  // name of the waveform file
    $dumpvars(0, memory_controller_tb);  // dump all signals in this module
    clk = 0;
    end
  always #5 clk = ~clk;

  // Instantiate memory controller
  memory_controller u_controller (
      .clk(clk),
      .rst_n(rst_n),
      .address(address),
      .wdata(wdata),
      .rdata(rdata),
      .read_enable(read_enable),
      .write_enable(write_enable),
      .ready(ready),
      .read_valid(read_valid),
      .write_done(write_done),
      .memory_address(memory_address),
      .write_data(write_data),
      .read_data(16'h1234),  // dummy memory input
      .r_e(r_e),
      .w_e(w_e)
  );

  initial begin
    #0 rst_n = 0;
    #20 rst_n = 1;

    $monitor("Time=%0t Addr=%h WData=%h RData=%h Ready=%b ReadValid=%b WriteDone=%b", $time,
             address, wdata, rdata, ready, read_valid, write_done);

    wdata = 16'h0000;
    address = 16'h0000;
    {write_enable, read_enable} = 2'b00;

    // Test multiple addresses
    repeat (2) begin
      #10 address = 16'h0010;
      #10 wdata = 16'hAAAA;

      repeat (4) begin
        #10{write_enable, read_enable} = {write_enable, read_enable} + 2'b01;
      end

      #10 address = 16'h0020;
      #10 wdata = 16'h5555;

      repeat (4) begin
        #10{write_enable, read_enable} = {write_enable, read_enable} + 2'b01;
      end
    end

    #50 $finish;
  end

endmodule
```

- TB output:

```
Time=20 Addr=0000 WData=0000 RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=30 Addr=0010 WData=0000 RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=40 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=65 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=0
Time=85 Addr=0010 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=90 Addr=0020 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=95 Addr=0020 WData=aaaa RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=100 Addr=0020 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=125 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=0 WriteDone=0
Time=145 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=150 Addr=0010 WData=5555 RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=155 Addr=0010 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=160 Addr=0010 WData=aaaa RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=185 Addr=0010 WData=aaaa RData=1234 Ready=0 ReadValid=0 WriteDone=0
Time=205 Addr=0010 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=210 Addr=0020 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=215 Addr=0020 WData=aaaa RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=220 Addr=0020 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=245 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=0 WriteDone=0
Time=265 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=275 Addr=0020 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
$finish called from file "testbench.sv", line 65.
$finish at simulation time                  310
```

Memory Controller Simulation Screenshots

- All controller signals:



Complete System RTL Code

```
module top_system (
    clk,
    rst_n
);
  input clk, rst_n;

  // CPU to controller signals
  wire [15:0] cpu_address, cpu_wdata, cpu_rdata;
  wire cpu_read_enable, cpu_write_enable;
  wire cpu_ready, cpu_read_valid, cpu_write_done;

  // controller to memory signals
  wire [15:0] mem_address, mem_write_data, mem_read_data;
  wire mem_r_e, mem_w_e;

  // instantiate CPU
  cpu u_cpu (
      .clk(clk),
      .rst_n(rst_n),
      .address(cpu_address),
      .wdata(cpu_wdata),
      .rdata(cpu_rdata),
      .read_enable(cpu_read_enable),
      .write_enable(cpu_write_enable),
      .ready(cpu_ready),
      .read_valid(cpu_read_valid),
      .write_done(cpu_write_done)
  );

  // instantiate memory
  memory u_memory (
      .clk(clk),
      .rst_n(rst_n),
      .address(mem_address),
      .write_data(mem_write_data),
      .read_data(mem_read_data),
      .r_e(mem_r_e),
      .w_e(mem_w_e)
  );

  // instantiate memory controller
  memory_controller u_controller (
      .clk(clk),
       .rst_n(rst_n),
       .address(cpu_address),
       .wdata(cpu_wdata),
       .rdata(cpu_rdata),
       .read_enable(cpu_read_enable),
       .write_enable(cpu_write_enable),
       .ready(cpu_ready),
       .read_valid(cpu_read_valid),
       .write_done(cpu_write_done),
       .memory_address(mem_address),
       .write_data(mem_write_data),
       .read_data(mem_read_data),
       .r_e(mem_r_e),
       .w_e(mem_w_e)
   );

endmodule
```

- Simple memory code:

```
module memory (
    clk,
    rst_n,
    address,
    write_data,
    read_data,
    r_e,
    w_e
);
  input clk, rst_n;
  input [15:0] address, write_data;
  input r_e, w_e;
  output reg [15:0] read_data;

  reg [15:0] mem[0:65535];

  always @(*) begin
    if (r_e) read_data = mem[address];
    else read_data = 16'b0;
  end

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      integer i;
      for (i = 0; i < 65536; i = i + 1) mem[i] <= 16'b0;
    end else begin
      if (w_e) mem[address] <= write_data;
    end
  end
endmodule
```

- Simple CPU code:

```
module cpu (
    clk,
    rst_n,
    address,
    wdata,
    rdata,
    read_enable,
    write_enable,
    ready,
    read_valid,
    write_done
);
  input clk, rst_n;
  output reg [15:0] address, wdata;
  input [15:0] rdata;
  output reg read_enable, write_enable;
  input ready, read_valid, write_done;


  reg [7:0] step;


  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      step <= 0;
      address <= 0;
      wdata <= 0;
      read_enable <= 0;
      write_enable <= 0;
    end else begin
      read_enable  <= 0;
      write_enable <= 0;
      case (step)
        0:
        if (ready) begin
          address <= 16'h0010;
          wdata <= 16'hAAAA;
          write_enable <= 1;
          step <= 1;
        end
        1: if (write_done) step <= 2;
        2:
        if (ready) begin
          address <= 16'h0010;
          read_enable <= 1;
          step <= 3;
        end
        3: if (read_valid) step <= 4;
        4:
        if (ready) begin
          address <= 16'h0020;
          wdata <= 16'h5555;
          write_enable <= 1;
          read_enable <= 1;
          step <= 5;
        end
        5: if (write_done) step <= 6;
      endcase
    end
  end
endmodule
```

Complete System TestBench Code

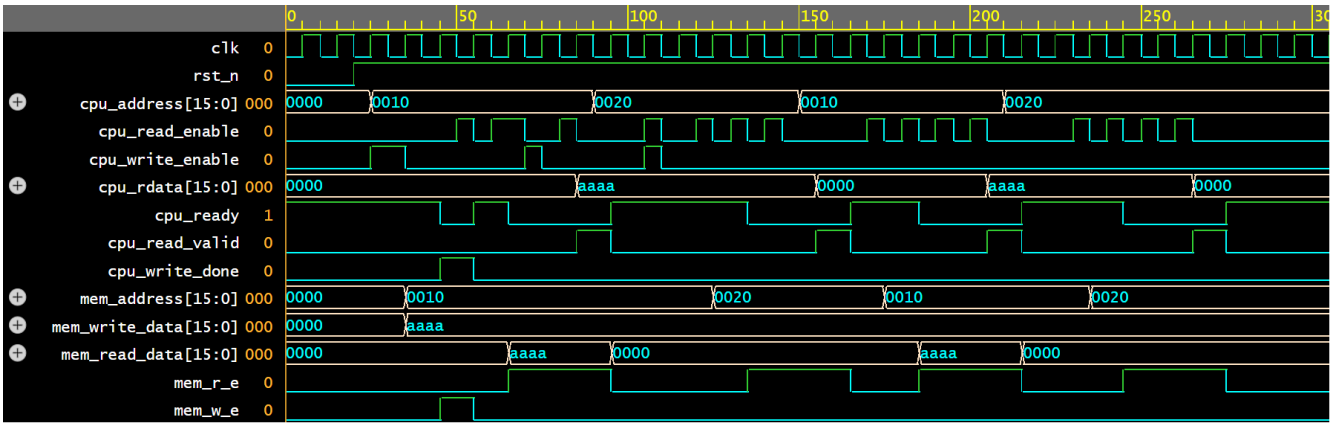
```
`timescale 1ns / 1ns
module top_system_tb;

  reg clk, rst_n;
initial begin
    $dumpfile("top_system_tb.vcd");  // name of the waveform file
    $dumpvars(0, top_system_tb);  // dump all signals in this module
    clk = 0;
    end
    always #5 clk = ~clk;

  top_system u_top (
      .clk  (clk),
      .rst_n(rst_n)
  );
  initial begin
    #0 rst_n = 0;
    #20 rst_n = 1;

    $monitor("Time=%0t Addr=%h WData=%h RData=%h Ready=%b ReadValid=%b WriteDone=%b", $time,
             u_top.u_cpu.address, u_top.u_cpu.wdata, u_top.u_cpu.rdata, u_top.u_cpu.ready,
             u_top.u_cpu.read_valid, u_top.u_cpu.write_done);

    {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} = 2'b00;

    // multiple addresses
    repeat (2) begin
      #10 u_top.u_cpu.address = 16'h0010;
      #10 u_top.u_cpu.wdata = 16'hAAAA;

      repeat (4) begin
        #10
        {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} =
                {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} + 2'b01;
      end

      #10 u_top.u_cpu.address = 16'h0020;
      #10 u_top.u_cpu.wdata = 16'h5555;

      repeat (4) begin
        #10
        {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} =
                {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} + 2'b01;
      end
    end
    #50 $finish;
  end

endmodule
```

- TB output:

```
Time=20 Addr=0000 WData=0000 RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=25 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=45 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=1
Time=55 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=65 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=0
Time=85 Addr=0010 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=90 Addr=0020 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=95 Addr=0020 WData=aaaa RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=100 Addr=0020 WData=5555 RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=135 Addr=0020 WData=5555 RData=aaaa Ready=0 ReadValid=0 WriteDone=0
Time=150 Addr=0010 WData=5555 RData=aaaa Ready=0 ReadValid=0 WriteDone=0
Time=155 Addr=0010 WData=5555 RData=0000 Ready=0 ReadValid=1 WriteDone=0
Time=160 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=1 WriteDone=0
Time=165 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=185 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=0
Time=205 Addr=0010 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=210 Addr=0020 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=215 Addr=0020 WData=aaaa RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=220 Addr=0020 WData=5555 RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=245 Addr=0020 WData=5555 RData=aaaa Ready=0 ReadValid=0 WriteDone=0
Time=265 Addr=0020 WData=5555 RData=0000 Ready=0 ReadValid=1 WriteDone=0
Time=275 Addr=0020 WData=5555 RData=0000 Ready=1 ReadValid=0 WriteDone=0
$finish called from file "testbench.sv", line 46.
$finish at simulation time                  310
```

Complete System Simulation Screenshots

- All controller signals:



Conclusion

In this project, we designed and verified a simplified memory controller that manages read and write operations between a CPU and memory. The controller was implemented using a basic FSM with idle, read, and write states, and it correctly prioritizes write operations when both read and write are requested. A simple CPU model was created to generate requests, and a memory module was added to complete the system. Testbenches were developed for both the controller and the full system to automatically check correctness through simulation. The results showed that the design behaves as expected, with proper handshaking using ready, read_valid, and write_done signals. Overall, the project met its main goals and provided hands-on practice with RTL design, simulation, and verification of digital systems.